TASK 1

1: Cybersecurity basics –

CIA triad: This is the core model for thinking about information security. Every security control exists to protect one (or more) of these three properties.

1: Confidentiality
Ensuring information is accessible only to those authorized to see it.

- Mechanisms: encryption (at rest and in transit), access control lists, authentication, need-to-know policies
- Violation example: an attacker intercepts unencrypted traffic and reads sensitive data (eavesdropping, data breach)

2: Integrity
Ensuring data is accurate, complete, and unaltered except by authorized action.

- Mechanisms: hashing (checksums, digital signatures), version control, audit logs, write-protection
- Violation example: an attacker modifies a bank transaction amount in transit, or tampers with a file undetected

3: Availability
Ensuring systems and data are accessible to authorized users when needed.

- Mechanisms: redundancy, backups, failover systems, DDoS protection, patching
- Violation example: a DDoS attack takes a website offline

Threat Types:

Threat	What it is	Primarily hits
Phishing	**Deceptive emails/messages tricking users into revealing credentials or installing malware**	**Confidentiality**
Malware	**Umbrella term for malicious software — viruses, worms, trojans, spyware**	**All three, depending on payload**
DDoS	**Overwhelming a system with traffic from multiple sources to knock it offline**	**Availability**
SQL Injection	**Inserting malicious SQL code into input fields to manipulate a database**	**Confidentiality, Integrity**
Brute Force	**Systematically guessing passwords/keys until one works**	**Confidentiality**
Ransomware	**Malware that encrypts data and demands payment for the decryption key**	**Integrity, Availability**

Attack Vectors:

These describe *how* an attacker gets in, as opposed to *what* they deploy once they're in.

1. **Social Engineering**
   Manipulating people rather than systems — pretexting, baiting, tailgating, phishing itself is often classified here too. The core insight: humans are frequently the weakest link, not the technology.
2. **Wireless Attacks**
   Exploiting Wi-Fi and other wireless protocols — rogue access points, evil twin attacks, WPA/WEP cracking, Bluetooth exploits, packet sniffing on unsecured networks.
3. **Insider Threats**
   Risk originating from people who already have legitimate access — employees, contractors, partners. Can be malicious (disgruntled employee exfiltrating data) or unintentional (an employee falling for phishing or misconfiguring a system). Often the hardest to detect because the activity looks "authorized."

2: Networking Basics

OSI Model — 7 Layers:

A conceptual framework for how data moves across a network. Mnemonic: **"Please Do Not Throw Sausage Pizza Away"**.

Layer	Name	Function	Examples
1	**Physical**	**Raw bit transmission over physical medium**	**Cables, hubs, radio signals**
2	**Data Link**	**Node-to-node delivery, MAC addressing, error detection**	**Ethernet, switches, ARP**
3	**Network**	**Logical addressing and routing between networks**	**IP, routers, ICMP**
4	**Transport**	**End-to-end delivery, reliability, flow control**	**TCP, UDP**
5	**Session**	**Establishing/managing/terminating sessions**	**APIs, sockets**
6	**Presentation**	**Data translation, encryption, compression**	**SSL/TLS, JPEG, ASCII**
7	**Application**	**User-facing services and protocols**	**HTTP, FTP, DNS, SMTP**

TCP/IP Protocol Suite

The practical, 4-layer model the internet actually runs on

TCP/IP Layer	Roughly maps to OSI	Key protocols
Application	**Layers 5–7**	**HTTP, HTTPS, DNS, FTP, SMTP, SSH**
Transport	**Layer 4**	**TCP, UDP**
Internet	**Layer 3**	**IP, ICMP, ARP**
Network Access	**Layers 1–2**	**Ethernet, Wi-Fi**

TCP vs UDP — the critical distinction:

- TCP (Transmission Control Protocol): connection-oriented, uses a 3-way handshake (SYN → SYN-ACK → ACK), guarantees delivery and order. Used where accuracy matters more than speed (web browsing, email, file transfer). The handshake itself is a common attack surface — a SYN flood exploits it by sending SYNs and never completing the handshake, exhausting server resources.
- UDP (User Datagram Protocol): connectionless, no handshake, no guaranteed delivery. Faster, lower overhead. Used where speed matters more than perfection (DNS queries, video streaming, VoIP, gaming). Its lack of a handshake also makes it easy to spoof — a major factor in DNS amplification DDoS attacks.

DNS & HTTP/HTTPS Deep Dive

DNS (Domain Name System) — translates human-readable domains into IP addresses.

1. Your device checks its local cache, then queries a recursive resolver (often your ISP or 8.8.8.8)
2. Resolver queries a root server → TLD server (.com, .org) → authoritative server for the domain
3. The IP address is returned and cached

Security-relevant DNS concepts:

- DNS spoofing/cache poisoning: injecting fake DNS records to redirect users to malicious sites
- DNS tunneling: exfiltrating data or establishing C2 channels by hiding it in DNS queries
- DNSSEC: adds cryptographic signatures to DNS responses to prevent spoofing

HTTP vs HTTPS

- HTTP (port 80): plaintext request/response protocol. Anything sent is readable by anyone intercepting traffic.

- **HTTPS (port 443)**: HTTP wrapped in TLS (Transport Layer Security), which provides:
    - Encryption — traffic can't be read in transit
    - Authentication — the server's certificate proves you're talking to who you think you are
    - Integrity — data can't be silently altered in transit

The TLS handshake (simplified): client and server agree on a cipher suite → server presents its certificate → a shared session key is derived (via asymmetric crypto) → subsequent communication is encrypted with that symmetric key. This is why TLS strips a huge amount of attacker capability compared to plain HTTP — no more trivial packet sniffing of credentials, for instance.

IP Addressing, Subnetting, and NAT

IP Addressing

- IPv4: 32-bit, written as four octets (e.g., 192.168.1.10), ~4.3 billion addresses (exhausted)
- IPv6: 128-bit, written in hex (e.g., 2001:0db8::1), astronomically larger address space
- Address classes/ranges matter for security: private (RFC 1918) ranges — 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 — are never routable on the public internet, so seeing one on the open internet is a red flag (usually misconfiguration or spoofing).

Subnetting
Dividing a network into smaller sub-networks using a subnet mask (or CIDR notation, e.g., /24).

- /24 = 255.255.255.0 = 256 addresses (254 usable) — a common small-office subnet
- Smaller subnets = smaller broadcast domains = better performance and easier containment of a breach (if one subnet is compromised, network segmentation limits lateral movement — a core defensive principle)
- Example: 192.168.1.0/24 covers 192.168.1.0–192.168.1.255, with .1.0 as the network address and .1.255 as the broadcast address

NAT (Network Address Translation)
Lets multiple devices on a private network share a single public IP address. Your router rewrites the source IP/port of outgoing packets and tracks the mapping so return traffic goes to the right internal device.

- Security side effect: NAT provides *incidental* obscurity (external attackers can't directly address internal devices) but it is not a substitute for a firewall — it doesn't inspect or filter traffic content, and it doesn't stop outbound-initiated attacks or malware phoning home.

3: Cryptography Basics

Symmetric vs Asymmetric Encryption

Symmetric Encryption
One shared secret key encrypts *and* decrypts data. Both parties must already possess the same key.

- **Pros**: fast, computationally cheap — ideal for encrypting large volumes of data
- **Cons**: key distribution problem — how do you securely get the key to the other party without an attacker intercepting it?
- **Algorithms**: AES (Advanced Encryption Standard — the current standard, typically AES-256), DES/3DES (legacy, considered weak/deprecated), ChaCha20 (used in some modern TLS ciphers)

Asymmetric Encryption (Public-Key Cryptography)
Uses a mathematically linked key *pair*: a **public key** (shared freely, encrypts) and a **private key** (kept secret, decrypts). What one key encrypts, only the other can decrypt.

- **Pros**: solves the key distribution problem — no need to pre-share a secret
- **Cons**: computationally expensive, much slower than symmetric encryption
- **Algorithms**: RSA, ECC (Elliptic Curve Cryptography — smaller keys, same security, more efficient), Diffie-Hellman (specifically for key *exchange*)

Why they're used together — hybrid encryption
This is the key insight: real-world systems (like TLS) don't pick one — they combine both. Asymmetric encryption is used briefly during the handshake to securely exchange a symmetric session key; then symmetric encryption (fast) handles the actual bulk data transfer. You get the security of asymmetric key exchange without the performance cost of using it for everything.

	Symmetric	Asymmetric
Keys	1 shared key	Public/private pair
Speed	Fast	Slow
Key distribution	Hard problem	Solved (public key is public)
Use case	Bulk data encryption	Key exchange, digital signatures, authentication

Hashing (MD5, SHA-256)

Hashing is **not encryption** — it's a one-way function. You can't "decrypt" a hash back to the original data; it's designed to be irreversible. Input of any size → fixed-length output (the "digest" or "hash").

Properties of a good cryptographic hash:

- **Deterministic**: same input always produces the same output
- **One-way**: computationally infeasible to reverse
- **Avalanche effect**: a tiny change in input drastically changes the output
- **Collision-resistant**: infeasible to find two different inputs producing the same hash

MD5 — 128-bit output. **Cryptographically broken.** Collisions can be deliberately generated relatively cheaply, meaning an attacker can craft two different files with the same hash. Still occasionally seen for *non-security* checksums (e.g., verifying a file wasn't corrupted in transfer), but should never be used for passwords, digital signatures, or anything security-critical.

SHA-256 — 256-bit output, part of the SHA-2 family. Currently considered secure and is the standard for most modern applications: blockchain (Bitcoin), TLS certificate fingerprints, file integrity verification, and (combined with salting) password storage.

Where hashing is used:

- **Password storage**: never store plaintext passwords — store a salted hash. (Note: for passwords specifically, purpose-built slow hashes like bcrypt, scrypt, or Argon2 are preferred over raw SHA-256, because they're deliberately slow to resist brute-forcing.)
- **Integrity verification**: comparing a downloaded file's hash against a published hash to confirm it wasn't tampered with
- **Digital signatures**: you hash the data first, then encrypt *the hash* with a private key (more efficient than encrypting the whole message)

A critical distinction to keep straight: encryption is reversible (with the right key); hashing is one-way by design. Confusing these two is one of the most common conceptual mistakes in security.

Digital Certificates & SSL/TLS

This is where symmetric encryption, asymmetric encryption, and hashing all come together in one real-world system.

The problem certificates solve: asymmetric encryption lets you send data securely to a public key — but how do you know that public key actually belongs to the server you think you're talking to, and not an attacker doing a man-in-the-middle attack?

Digital Certificates
A certificate binds a public key to an identity (a domain name, organization, etc.) and is digitally signed by a trusted **Certificate Authority (CA)** — e.g., DigiCert, Let's Encrypt. Your browser trusts a pre-installed list of root CAs, and trust flows down a **chain**: Root CA → Intermediate CA → the site's certificate.

A certificate typically contains: the domain name, the public key, the issuing CA, validity dates, and the CA's digital signature over all of it.

The TLS Handshake, step by step:

1. **Client Hello**: browser initiates, states supported TLS versions and cipher suites
2. **Server Hello + Certificate**: server responds, presents its certificate
3. **Certificate validation**: browser checks the certificate chain up to a trusted root CA, checks it hasn't expired or been revoked, and checks the domain matches
4. **Key exchange**: using asymmetric crypto (often via Diffie-Hellman for "forward secrecy"), client and server derive a shared symmetric session key
5. **Symmetric encryption begins**: all further communication uses that session key (fast, bulk encryption)

Why this design matters for security:

- Asymmetric crypto solves *authentication* (proving identity) and *key exchange* (safely establishing a shared secret)
- Symmetric crypto handles the *actual data encryption* (efficient at scale)
- Hashing secures the *certificate's signature itself* (the CA hashes the certificate contents, then encrypts that hash with its private key — anyone can verify it using the CA's public key)

Common certificate-related attacks: expired/self-signed certs (browsers warn, but users often click through), certificate spoofing, and SSL stripping (downgrading a connection from HTTPS to HTTP mid-handshake — mitigated by HSTS, which forces browsers to only ever connect via HTTPS to a given domain).